

Lab 2: Sensor/Actuators Bring-up

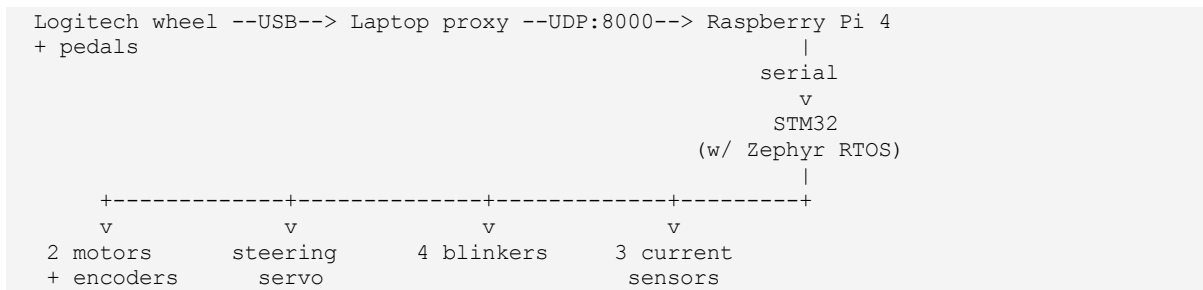
Release: 09/18/2026

Due: 10/01/2026, 11:59pm

Version 1.0

What you are building

In this lab, you'll be wiring up the sensor and actuators on your car, sanity checking that they all work, creating their control code with (Zephyr) RTOS, and interfacing with the Logitech sim racing steering wheel. The steering wheel plugs into your laptop, a proxy on the laptop streams state to the Pi over UDP, the Pi hands commands to a single STM32 over a serial link, and the STM32 drives everything on the car.



The steering wheel does **not** physically connect to the Pi. The steering wheel connects to your laptop which runs the Logitech SDK and the SDL fallback. This eventually allows you to make the link wireless.

What is not in this lab / deviates from the spec in your Lab 1 writeup is as follows:

- **No CAN, and no second STM32.** Every sensor and actuator runs on one STM32. The Pi-to-STM32 serial link stands in for the bus. You will later be splitting this into two STM's.
- **No force feedback on the steering wheel.** You will not be transmitting the steering servo current up the UDP stream to apply a force to the steering wheel.
- **No PCB.** You have breadboards and jumper wire to connect everything up. This does not mean but unorganized. We've given you everything to stay (reasonably) neat.

Disclaimers

On AI. You should always be ready for questions that demonstrate that you understand your system and code. AI is allowed in this class, but we also want you to explore the most effective ways to use it. Some good uses of AI might look like *"here is the H-bridge datasheet section on IN1, IN2 and ENA - explain what each input combination does to the motor, then ask me follow up questions"* or *"review this ISR for anything that should not be in an ISR."* Bad uses look like *"write me a controller scripts"* and *"how do I wire the H-bridge to the STM32."* (without more significant context). Ask AI to help you work through details, not think for you.

On part and components. Feel free to reach out to course staff (your TA is your primary point-of-contact) if you need help getting auxiliary components that you feel could support you.

The handout itself. This handout deliberately provides high-level design directions opposed to low-level details: one of the learning objectives for this course is to enable students to think critically about different design choices. That is the shape of the course - you find the datasheets, choose the workflow, and defend the choice, while doing your own research and learnings. **You do not need to write the literal answer down for most questions.** Refer to the [deliverables](#) section to know exactly what you need to prepare for checkoff and writeup. Please read the entire document before you start.

What is in the kit

Hardware

Part	Why you have it
Raspberry Pi 4, official USB-C supply, microSD card	The cockpit zone. Receives the UDP stream and commands the STM32. Use only the supplied adapter.
STM32 Nucleo + Mini-B USB cable	The one MCU for this lab. Runs Zephyr and drives every actuator.
Car chassis, 2 DC motors with encoders, steering servo	The drivetrain and steering. The encoders are your only velocity feedback.
Logitech steering wheel, pedals, 24 V brick	Cockpit input, connected to your laptop. The power brick is required - the steering wheel will not come up on USB power only.

Electronics

Part	Why you have it
Breadboards, large and small	Large for the build, small for breakout circuits
Jumper kits, pre-bent jumpers, spools of 22 AWG solid wire	Wiring components/electronics together
12 V wall adapter + barrel-jack screw-terminal splitters	Motor supply for the H-bridge.
Buck converter and 5 V linear regulator	Step 12 V down for the servo or any other intermediate rail.
Motor driver (dual H-bridge)	Direction, PWM, and the braking mode you need for S2.
LEDs and resistors	Front and rear left/right blinkers.
Current sensors	One for each motors, one on the servo.

Part	Why you have it
USB-C multiport hub (Ethernet, HDMI, microSD)	Flashing the SD card and getting the Pi on a wired LAN for SSH.
Mini-HDMI to HDMI adapter, Ethernet cable	For the Pi, if you go the monitor route or need wired networking.
Small Phillips and flat screwdrivers	For every screw terminal on the splitters, buck converter and H-bridge.
Ziploc bags and a bin	Keep your parts together. You will be carrying this around.

There is no dedicated push button in the kit. Use one of the wheel's buttons as the self-test input - there are plenty of them, and you are already decoding the button array. Anything else you need, see the disclaimer about being thrifty and ask the course staff.

About your Lab 1 design document: treat it as a living document for the rest of this class. You should be updating and keeping it looking good 'till the end. As you discover what numbers change, or perhaps there was an error in your topology, etc etc, it should change there too. We may ask for it throughout the course. Make sure to keep a Revision history at the end of your document for major changes.

Part 0: Development environment

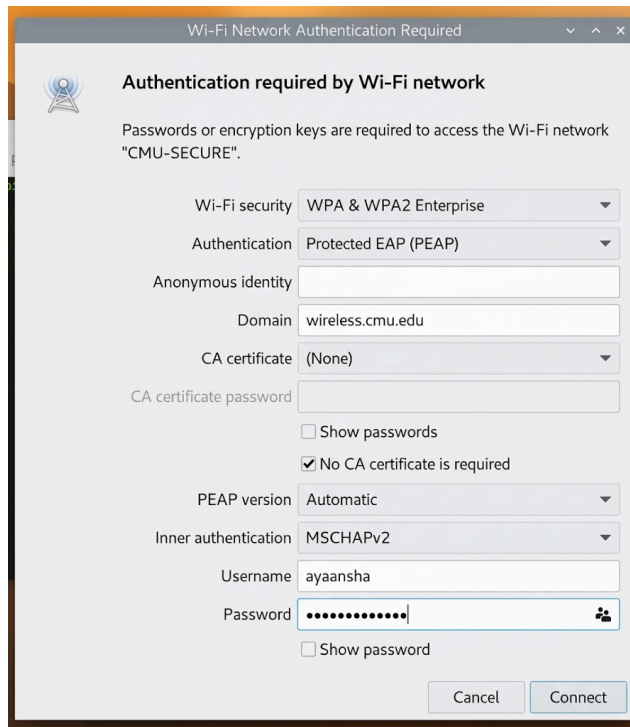
Raspberry Pi. Flash Raspberry Pi OS (64-bit, full desktop image) with Raspberry Pi Imager (<https://www.raspberrypi.com/software/>). Open the Imager's settings gear before you write the card - hostname, username, password, SSH, Wi-Fi and locale all get set there, and skipping it means you need a monitor to finish setup. The USB-C hub in your kit handles both the SD card and wired Ethernet.

How you get a shell on the Pi is up to you:

- Raspberry Pi Connect (<https://www.raspberrypi.com/documentation/services/connect.html>) is the least painful route. Make an account, give the key during flashing, and you get a remote shell and screen with no keyboard, video and mouse (KVM) setup at all.
- KVM for first boot, then SSH over the local IP for everything after. You have a micro-HDMI adapter; the monitor, keyboard and mouse are on you.
- Ethernet to a router you control, if campus networking fights you.

On CMU-SECURE

(<https://www.cmu.edu/computing/services/endpoint/network-access/wireless/how-to/connect.html>), set the CA certificate to none. Campus Wi-Fi may block mDNS, so `wheelpi.local` will not resolve, and it may block device-to-device traffic entirely - which later looks like a broken proxy rather than a network policy. `ping` the Pi from your laptop before you debug anything else.



This is what your wifi setup should look like.

Power the Pi from its own supply, ONLY. Brownouts show up as random reboots, dropped SSH sessions and corrupted SD cards, every one of which looks like a software bug and none of which is. `vcgencmd get_throttled` should return `0x0`.

Zephyr. Follow the Zephyr getting-started guide

(https://docs.zephyrproject.org/latest/develop/getting_started/index.html) to install `west`, the SDK and the toolchain. Windows and Linux hosts are known good. macOS is documented and does work, but it is known to be finicky - if you are on a Mac and fighting the toolchain, moving to a Linux VM is reasonable. Background reading and video is in Appendix B.

Checkpoint. You can SSH into the Pi, `vcgencmd get_throttled` returns `0x0`, and you have built and flashed a blinky onto the Nucleo from Zephyr. Do not wire anything to the STM32 until blinky runs.

Part 1: Get steering wheel communicating with the Pi

One sanity check: wheel state, printed live on the Pi. Nothing on the car moves yet.

Unboxing. Clamp the wheel to a desk before you power it - Logitech's quick-start guide

(<https://www.logitech.com/assets/65932/2/g923-racing-wheel-qsg.pdf>) covers assembly. Power brick first, then USB. On power-up the wheel sweeps to full left and full right to calibrate, so keep hands and cables clear. A solid LED means it has power; a slowly blinking one means it does not, so check the connection at the wheel, at the brick and at the wall.

Run the proxy and the monitor. Full steps are in Appendix A. In outline: the proxy runs on your laptop and streams to the Pi's IP on UDP port 8000, and the monitor runs on the Pi and prints each

packet. The Mac and Windows proxies send byte-identical packets, so the Pi side does not care which you use.

Record these values - they feed everything downstream, and the team-defined values table:

What	Record
Steering axis	Raw value at full left, centered, full right
Throttle axis	Raw value released and fully pressed
Brake axis	Raw value released and fully pressed
Buttons	The raw index of each button you plan to use

Ranges and button indices differ between the Mac and Windows proxies. That is not a bug, and finding out what your setup actually reports is the point of this step. Pick one proxy and stay on it, or handle both explicitly.

Checkpoint. The packet counter on the Pi climbs steadily, steering swings through its full range, both pedals move, and you have written down the index of every button you care about.

Part 2: The Pi-to-STM32 link

About the guiding questions. From here on, each part comes with a set of questions. You do not have to answer them in a write-up - they are there to point you at what to read and what to figure out. Only the deliverables and the checkoff at the end are graded. That said, having answers makes debugging with us a lot faster.

The Pi decodes UDP packets and hands the STM32 something it can act on. You design that something. We recommend UART; USB CDC also works. Whatever you choose, document it, because this link is a rehearsal for the CAN message catalog you already wrote in Lab 1.

What the link has to do, however you build it:

- Carry a command to the STM32 on every UDP state update, and no less often than every 50 ms.
- Carry a status frame back from the STM32 every 20 ms $\pm 10\%$, including current-sensor readings and the zone's state. This is your heartbeat.
- Let the STM32 notice the link is gone. Three consecutive missed command updates (150 ms) means fail safe.
- Let the receiver reject a malformed or out-of-range frame rather than acting on it.

Questions to help guide writing the parser:

- What fields does the command actually need? Are you sending raw wheel counts or something already scaled, and who does the conversion?
- How does a receiver that joins mid-stream, or drops a byte, get back in sync? What does your framing look like?
- How do you tell a truncated frame from a valid one?

- What happens if the Pi sends faster than the STM32 drains the buffer? Who blocks, who drops, and which do you want when the stale data is a throttle command?

Wiring note. Pi GPIO is 3.3 V and is not 5 V tolerant - 5 V on a Pi pin kills the pin or the Pi. If you use the Pi's GPIO UART, you have to free it from the serial console first.

Checkpoint. Turn the wheel and a value changes in a print statement on the STM32. Unplug the cable and the STM32 notices within 150 ms.

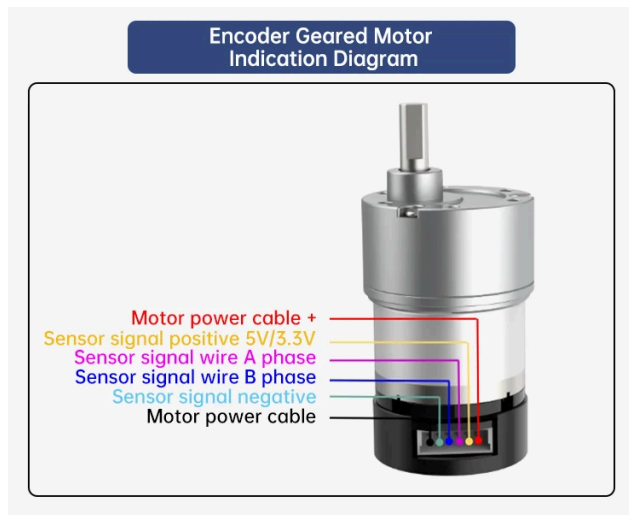
Part 3: Car sensors and actuators

If needed, you can find specifications and pinouts of your car's components here:

<https://www.hiwonder.com/products/ackermann-steering-chassis?variant=40382428348503>

You do not need to use the included Encoder motor driver controller.

3.1 Motors and encoders



Goal. Throttle position sets a target wheel velocity through a documented, monotonic mapping, held by closed-loop control on encoder feedback. Full throttle corresponds to a maximum velocity you choose and document. Velocity for control purposes is the average of the two encoders.

Questions.

- What are the wires coming out of your motor?
- The encoder gives you transitions. How do you turn a stream of transitions into a velocity?
- How does the L298N H-bridge chip work? What do you need to be careful about?
- What is a PID controller, which terms do you need here to calibrate, and how do you implement it on a microcontroller?

Sanity check. Spin each wheel by hand and confirm the count moves, and moves the other way in reverse. Then change the target velocity and watch the measured velocity change and coverage at a stable number. Then with your hand, apply some resistance to the wheel; the pid controller should push back and apply extra power to the motor.

3.2 Brake and dynamic braking

Goal. Pressing the brake pedal means motor PWM disabled and the H-bridge in dynamic braking. Brake takes priority over throttle under all circumstances.

Questions.

- What states can an H-bridge put a motor in? What is the difference between letting it coast and dynamic braking?
- Which input combination gets you each state on *your* driver module?

Sanity check. Get the wheels up to speed, press brake, and compare how fast they stop against simply cutting power. Scope the direction pins and confirm you got the state you think you commanded.

3.3 Steering servo

Servo pins are usually color coded where red and black/brown are +5V and ground and white or orange take a 5V PWM input signal.

Goal. The servo angle tracks the cockpit wheel angle over the full mechanical range of the chassis steering, through a documented, monotonic mapping.

Questions.

- What signal does yours expect: pulse width range, period, and what does it do with values outside that range?
- How far does the chassis linkage move before it reaches its maximum?
- Where does the servo get its power, and ***it's NOT the MCU board!***

Sanity check. Full left, center, full right, back. The linkage reaches both ends without buzzing or straining. Buzzing at an endpoint means you are stalling the servo.

3.4 Blinkers

Goal. Left and right turn signals from the wheel buttons, blinking at 1 Hz $\pm 10\%$ with a 50% duty cycle. One side cancels the other. A blinker self-cancels when the wheel passes a turn threshold and returns back past it. In the error state, all four flash together at 2 Hz as hazards.

Questions.

- Where does the blink period come from - a timer, a thread, something else? What keeps it from drifting against your other periodic work?
- Can you draw the blinker state machine with every transition labeled? What state are you in after pressing right, turning right past the threshold, then pressing left?
- What is your turn threshold in wheel units, and how did you pick it?

Sanity check. Scope two blinker outputs at once and measure period and duty with cursors, over several periods rather than one.

3.5 Current sensors

Goal. Read all three and report them in the status frame. Read-only this lab - nothing acts on the values yet - but the numbers you record become the fault thresholds later.

Questions.

- What does the sensor output, and how do you get from that to amps?
- What is your ADC's reference and resolution, and what current is one LSB?
- How noisy is the reading with PWM running nearby, and what does filtering cost you?
- What reading means a motor is disconnected? What means it is stalled? How far apart are those in practice?

Sanity check. Record each sensor at rest, free-spinning, and briefly stalled. Three numbers per sensor, written down. Do not hold a stall for more than a second or two.

Part 4: RTOS Integration

Everything in Part 3 eventually needs to be scheduled by Zephyr, on a priority-based preemptive scheduler.

It is best to bring up everything in Zephyr RTOS while you are testing each component/subsystem. If you started on bare metal or with simple main loops, now is your opportunity to pull these functions together using Zephyr.

Questions.

- What belongs in a thread, a work item, a timer callback, and an ISR? Those are four different tools and you have work that fits each.
- What is the period and the deadline of each piece of periodic work - control loop, blinker, heartbeat, serial receive, ADC? Write them in a table.
- What preempts what? Brake has a 2 ms budget and the blinker has 100 ms. Do your priorities reflect that, and can you point at the line where you set them?
- How does state move between threads without tearing? `volatile` is not a synchronization primitive - what is, in Zephyr?
- Is anything blocking, logging, or allocating inside an ISR?

Start from the lecture slides on RTOS scheduling, then go to the Zephyr material in Appendix B. Devicetree and Kconfig are where most people lose a day - read about them before you need them, not during.

Deliverable tie-in. The task table - name, period, priority, deadline, what it talks to - is part of what you hand in, and it is what we walk through at checkoff.

Part 5: Power and wiring

No wiring diagram from us - working out what connects to what is part of the lab. We do want you to finish the semester with the boards you started it with, so read this.

Rail	Source	Feeds
12 V	Wall adapter, through the barrel-jack splitter	H-bridge motor supply
Stepped-down	Buck converter or linear regulator off the 12 V	Servo, and any other intermediate rail
Board rails	STM32 from your dev machine's USB; Pi from its own supply	The MCUs, and sensors within their current limits

Questions.

- What is ground here? You have four or five things on this bench with their own power sources. What has to be true about their grounds for any signal between them to mean anything?
- Why does the servo not run off the MCU board, and what happens to the 3.3 V rail if you try?

Never power the Pi from the STM32 or the STM32 from the Pi. Keep motor wiring physically away from encoder and sensor wiring. Use the small breadboards to keep subsystems separable - you will be pulling this apart to move it.

How to destroy an STM32

All of these are quick, quiet and permanent:

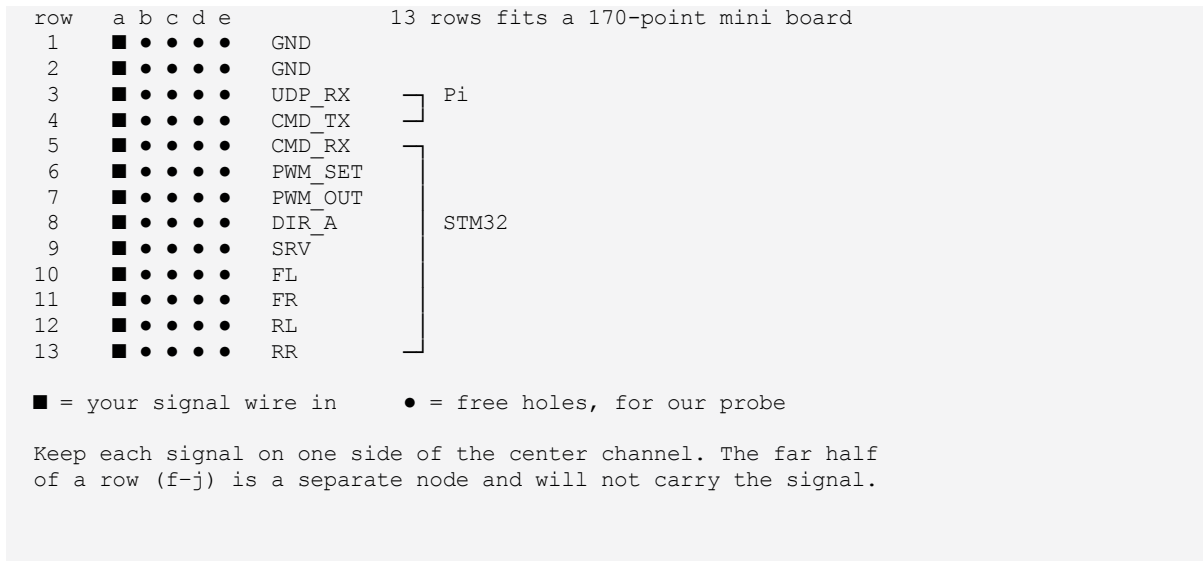
- **5 V on a pin that is not 5 V tolerant.** Many are, some are not, and the analog pins usually are not. Check the datasheet before you connect a current sensor output.
- **Drawing too much current out of a pin.** LEDs need series resistors. A servo or motor never gets driven from a GPIO, however small it looks.
- **Reversing polarity, or bridging a rail to ground with a stray jumper.** Count your rows twice before you power up.
- **Feeding the board from USB and an external 5 V at once,** without checking the board's power-selection jumper.

How you can tell you did: the board stops enumerating over USB, the programmer cannot find the target, the chip is hot, the 3.3 V rail measures low, or one GPIO is stuck while the rest behave. Power down, measure the rails against ground, find the short, and tell us - we would rather know now than at checkoff.

Test points

Grading in the lab is heavily based on timing which we cannot inspect without probe points.

Requirement: bring every test point below out to one of the small breadboards, one signal per row, going down the board in the order shown. The five holes in a row are tied together, so each row is exactly one node - two signals in the same row is a short. Label the board on a strip of tape beside it. We will clip onto this board and nothing else.



Point	Board	Toggles on
UDP_RX	Pi	Reception of each UDP state update
CMD_TX	Pi	Transmission of the command to the STM32
CMD_RX	STM32	Reception of that command
PWM_SET	STM32	Writing a new motor duty cycle
PWM_OUT	STM32	The motor PWM output itself
DIR_A	STM32	H-bridge direction input
SRV	STM32	Servo control signal
FL FR RL RR	STM32	The four blinker outputs

Measuring a duty-cycle change. A throttle response ends in a duty-cycle change, and a duty cycle is not a single edge - there is no one moment to put a cursor on. So instrument it: toggle `PWM_SET` in the same code path, immediately after you write the new value to the timer. Then channel 1 on `CMD_RX`, channel 2 on `PWM_SET`, trigger on `CMD_RX`, and you have the software response time directly. Separately, scope `PWM_SET` against `PWM_OUT` to show the waveform actually changed after that point. Report both, and be clear about which part is software and which is the actuator.

Brake is easier - the direction pins change on a single edge, so `CMD_RX` to `DIR_A` is a straight two-cursor measurement.

Blinker tolerances. Measure period and duty on the blinker points with cursors or automatic measurements, averaged over several periods. A single period tells you nothing about drift.

Which Lab 1 requirements apply

The timing numbers are the same as Lab 1.

Lab 1 requirement	This lab
R1 - priority-based preemptive RTOS	Yes.
R2.1 / R2.2 - throttle and brake within 2 ms	Yes, from the STM32 test point that communicates to the PWM or H-bridge change.
R2.3 - steering within 50 ms	Yes (BUT drop the force-feedback portion for this lab). Test same way as R2.1 and R2.2
R2.4 - blinkers within 100 ms, 1 Hz $\pm 10\%$, 50% duty	Yes. Both sides share one MCU and one time base here, so the 1 ms front/rear sync is nearly free - it gets hard once the sides are on different zones.
R2.5 - test button, local fail within 10 ms, debounced	Yes, using a wheel button.
R3 - error state	Partial. Fail-safe outputs and hazards on: self-test press, loss of the Pi link, or an out-of-range command. Power-up still starts in the error state.
R4.1–R4.4, R4.6, R4.7 - CAN bus, catalog, redundancy	No. Your Pi-to-STM32 link is the stand-in, and you still define and document its message format.
R4.5 - heartbeat and fault detection	Substituted: a status frame from the STM32 every 20 ms $\pm 10\%$, and three missed command updates from the Pi means fail safe.
R4.8 - fault-injection jumpers	No. Unplugging the serial cable is this lab's fault injection.
S3 - force feedback	No. The cross-platform proxy has no force-feedback path.
S6 - CAN bus fault tolerance	No. Nothing to be tolerant of yet.

Checkoff

Run the following sequence for a TA with your system in front of them.

1. **Cold start.** Power up with the proxy stopped: the system starts in the error state, hazards flashing, motor braked. Start the proxy; once valid commands arrive it goes to normal operation.
2. **Steering.** Full left, center, full right. The servo tracks monotonically across the full linkage range with no buzzing at the endpoints.
3. **Throttle.** Quarter, half, full. Velocity rises and holds. We will lightly load a wheel and watch the controller recover.
4. **Brake.** At speed, press brake - the wheels stop in braking mode, visibly faster than a coast. Then hold throttle and brake together: brake wins.
5. **Blinkers.** Left, right, switch between them. Turn past the threshold and back to self-cancel. We will measure one blinker's period and duty on the scope.
6. **Self-test.** Single press of your chosen wheel button: hazards and braked motor within 100 ms. Double press: back to normal.
7. **Link loss.** We unplug the Pi-to-STM32 cable. Fail-safe within 100 ms. Reconnect, and it recovers.
8. **Bad input.** Show us the system refusing an out-of-range command. How you inject it is up to you.
9. **Current sensors.** Live readings at rest, running, and briefly stalled.
10. **Timing.** Your scope captures for throttle, brake, steering and blinker response, measured as described above.

Expect verbal questions throughout, directed at everyone on the team and not just whoever wrote the code for a particular part. They will be about the design decisions in Parts 2 through 5 - the guiding questions are a fair preview of the territory, including how you handled the mechanical buttons

Deliverables

This course is demo-driven. Three quarters of the grade is the system working in front of us and you being able to explain it. The written pieces are short on purpose.

Demo	Points
Live demo - the checkoff sequence above	55
Checkoff questions, answered by the team	20
Writeup (to be submitted to Gradescope)	—
System (block) diagram of what you built this lab	4
Circuit schematic of the dev kit, including power rails and ground	6
Updated team-defined values table	4
Task table - name, period, priority, deadline	4
Brief Integration writeup: how this becomes three zones on CAN	7

The **schematic** has to be complete enough that someone could rebuild your wiring from it. Neat by hand is acceptable; a capture tool is better practice, and it's good practice for the PCB (some of the connections will transfer over!)

The **integration writeup** should max be one page. Just show that you have thought about what next steps you'll be taking for integrating 3-zone dual-CAN. Which functions move to which zone, which of your in-memory variables become CAN messages, what IDs and priorities you would give them, and what breaks when the blinker halves end up on different MCUs with different clocks.

Appendix A: Streaming the wheel to the Pi

Code files can be found at the following github repo:

<https://github.com/RockyShaikh/logitech-wheel-dev-updated-f26>. Please follow the instructions in WHEEL_STREAM_LAB.md for setup instructions. The proxy is compatible with both mac and windows, but each one has different instructions. Clone the repo onto whatever your proxy machine is.

Packet format. Each packet is 276 bytes: a 4-byte little-endian counter followed by a 272-byte `DIJOYSTATE2` struct (the Windows DirectInput layout, defined in `proxy_receiver/state.h`). The receiver only reads three fields from it - `lX` for steering, `lY` for throttle, `lRz` for brake - plus the button array. The macOS and Windows proxies emit byte-identical packets, so the Pi side is unchanged either way.

What runs where.

Machine	Files	Needs
Raspberry Pi	<code>proxy_receiver/wheel_monitor.c, state.h</code>	gcc
Mac laptop	<code>mac_proxy.py</code>	Python 3, pygame-ce
Windows laptop	<code>proxy_gui.py, logitech_steering_wheel/</code>	Python 3, PyQt5, Logitech G HUB or Gaming Software

The laptop and the Pi must be on the same network - both on CMU-SECURE, both on the same home Wi-Fi, or joined by an Ethernet cable.

On the Pi. Find the address with `hostname -I` (check it again whenever things stop working; campus addresses change). Copy the two files over, build with `gcc -O2 -Wall -o wheel_monitor wheel_monitor.c`, and run `./wheel_monitor -r`. It prints that it is listening on port 8000 and then reports no packets once a second, which is correct until a proxy starts. `-r` prints the raw index of each pressed button; `-n <N>` prints every Nth packet if the output is too fast to read.

On a Mac. Create a virtualenv and `pip install pygame-ce` - the maintained fork, still imported as `pygame`; plain `pygame` has no build for current Python versions. Run `python3 mac_proxy.py --probe` first and move one control at a time to learn which axis index is which. Then:

```
python3 mac_proxy.py --remote <pi-ip> --steer <n> --throttle <n> --brake <n> --interval-ms 50
```

Add `--invert-throttle` or `--invert-brake` if a pedal read `-1` at rest. Allow the incoming-connections prompt if macOS shows one. The `force --` in the status line is expected; nothing sends force back.

On Windows. Install the Logitech software before plugging in the wheel - if Windows installs its own driver first, the SDK will not see the wheel and you will need to uninstall that driver and reinstall the Logitech software with the wheel still connected. Then `pip install PyQt5`, edit the `## Configure this ##` block in `proxy_gui.py` so `REMOTE_HOST` is the Pi and `LOCAL_HOST` is this laptop's own IP, run it from the repo folder, and click **connect**. A wrong `LOCAL_HOST` produces an `OSError` on startup. The constant `No data` in the console is expected.

What you should see on the Pi. One line per packet with the packet number, the raw `steer / thr / brk` values, and dashes or names for the buttons. Button *names* in the monitor are provisional - they use commonly cited Windows indices that have not been confirmed on this setup, and the Mac proxy passes SDL's own numbering through, which differs. The `raw:` indices are the ground truth. Record those.

Appendix B: Zephyr background

Zephyr has a real learning curve, and most of it is front-loaded: the build system, Kconfig and devicetree all show up before you write anything interesting. Start with the lecture slides on RTOS scheduling, then pick one of these and work through it rather than reading all three.

Resource	Good for
Zephyr getting started - https://docs.zephyrproject.org/latest/develop/getting_started/index.html	Toolchain install, first build, first flash. Do this one regardless.
Zephyr project documentation - https://docs.zephyrproject.org/latest/	Reference for kernel services (threads, timers, work queues), the PWM and GPIO APIs, and devicetree. Where you will live once you are building.
Nordic Developer Academy - https://academy.nordicsemi.com/	Courses on using nRF Connect SDK which is built around Zephyr. Note that nRF Connect SDK will not be used, but much of the content in the nRF Connect SDK Fundamentals and nRF Connect SDK Intermediate courses apply.
DigiKey's Introduction to Zephyr video series - https://www.digikey.com/en/videos/d/digi-key-electronics/introduction-to-zephyr-part-1-getting-started-installation-and-blink-digikey	A 12-part walkthrough covering CMake, Kconfig, devicetree, drivers, threading and GPIO interrupts. Written companion and code repository at https://github.com/ShawnHymel/introduction-to-zephyr .

The three concepts worth understanding before you fight them: **devicetree** describes your hardware and is how you name a pin or a PWM channel; **Kconfig** (`prj.conf`) turns subsystems on and off; **west** is the meta-tool that drives builds and flashing. A large fraction of “my board does nothing” is one of these three, not your application code.

Appendix C: Things that often go wrong

Symptom	Usually means
Pi reboots at random, SSH drops, SD card corrupts	Inadequate power. <code>vcgencmd get_throttled</code> should return <code>0x0</code> . Fix this first; it masquerades as every other bug.
<code>wheelpi.local</code> does not resolve	Campus Wi-Fi blocking mDNS. Use the IP from <code>hostname -I</code> or your router's client list.
Monitor prints <code>no packets</code> forever	Wrong Pi address in the proxy, laptop and Pi on different networks, or the campus network blocking device-to-device traffic. <code>ping</code> the Pi first.
<code>bind</code> fails on the Pi	Something already holds port 8000 - usually a second monitor or the upstream receiver.
Occasional <code>packet(s) lost</code>	Normal on Wi-Fi. Constant loss means switch to Ethernet.
No joystick found on the Mac	The wheel's power brick is not connected, or you are going through a hub or a charge-only cable. Plug straight into the laptop.
Probe runs but all axes read 0	The wheel has not finished calibrating.
No matching distribution found for <code>pygame</code>	You installed <code>pygame</code> instead of <code>pygame-ce</code> .
<code>ModuleNotFoundError</code> for <code>pygame</code> or <code>PyQt5</code>	The virtualenv is not active in this shell.
Windows proxy connects but the Pi sees nothing	Wrong <code>REMOTE_HOST</code> , different networks, or the firewall blocked Python.
Encoder counts garbage, ADC drifts	Almost always a missing common ground.
Servo buzzes and gets hot	It is stalling against a mechanical stop. Clamp your commanded range.
Motors stutter under load	Supply sag. Check that the motor rail is the 12 V supply and not something borrowed from logic.
STM32 stops enumerating after a wiring change	See "how to tell you destroyed it" in Part 5.

Revision History:

9-18-26 Version 1.0: Initial release